

Sistema Emalog

O que foi feito em 17 de setembro de 2026

Resumo

A produção deixou de perder dados, o WhatsApp voltou a funcionar num servidor da própria empresa e as conversas passaram a ser atendidas ao lado do Kanban. Em resumo, cinco frentes:

- **Banco de produção.** O PostgreSQL no Amazon RDS entrou no ar e o sistema já grava nele. Antes, cada publicação apagava tudo o que havia sido cadastrado.
- **Trava contra SQLite.** O sistema passou a recusar subir sem o PostgreSQL quando está na AWS, em vez de cair para um banco vazio sem avisar. *Falta publicar.*
- **Evolution própria.** O WhatsApp saiu do servidor de terceiros, que havia caído, e roda num servidor da conta da empresa. Mensagem de teste recebida com sucesso.
- **Conversas junto do Kanban.** A aba Conversas da Central de Contratação mostra a Central de Atendimento de verdade. *Falta publicar.*
- **Chatwoot desativado.** Saiu do código, da configuração e da política de segurança.

1. Banco de produção no PostgreSQL do RDS

O problema. A produção rodava em SQLite, num arquivo dentro da pasta da aplicação. O Elastic Beanstalk substitui essa pasta a cada publicação, então tudo o que fosse cadastrado entre dois deploys se perdia no seguinte. Isso já acontecia antes da Central de Atendimento.

O que foi feito. O script `criar_banco_rds.sh` rodou no AWS CloudShell e criou o banco `ema-log-db`: PostgreSQL 18.3, sem acesso público, criptografado, protegido contra exclusão, acessível apenas pelo sistema. O endereço do banco foi configurado no ambiente.

O erro que apareceu no meio do caminho. O script terminou com "Waiter EnvironmentUpdated failed: Max attempts exceeded". Não era falha do banco nem do ambiente: o comando da AWS que acompanha o reinício desiste depois de seis minutos e meio, e o ambiente ainda estava atualizando. O banco já estava criado e a configuração, aceita. Bastou esperar, conferir os eventos e rodar o script de novo, que reaproveitou tudo.

Confirmado. Em `/conversas/api/status`: `"dialeto": "postgresql"` e `"schema_ok": true`. As tabelas, colunas e índices da Central estão todos no banco novo. O que estava no SQLite antigo não foi copiado: o banco começou vazio.

2. Trava contra SQLite no Elastic Beanstalk

Ter o banco certo configurado não bastava. Se o PostgreSQL não respondesse em três segundos durante a inicialização, o sistema subia com um SQLite vazio e só registrava um aviso no log — ou seja, voltaria a gravar num banco que some.

Agora, quando está na AWS, o sistema tenta o PostgreSQL dez vezes, por cerca de um minuto, e se ainda não conseguir, para de propósito, com a explicação no log. O site fica fora do ar em vez de aceitar cadastros que seriam perdidos. No computador, para desenvolvimento, nada muda: continua usando SQLite.

O campo `"trava_sqlite"` foi acrescentado ao `/conversas/api/status` para dar como conferir, depois da publicação, que a trava está valendo de verdade na máquina de produção.

Defeito antigo corrigido de passagem. O sistema decidia se o banco era SQLite procurando a palavra "sqlite" em qualquer parte do endereço. Um banco de teste chamado `r_trava_sqlite` foi tratado como SQLite e não subiu. Passou a olhar só o começo do endereço. A produção nunca foi afetada.

3. Evolution própria na AWS

O problema. O servidor que atendia o WhatsApp, `evolution-evolution.iqutxq.easypanel.host`, parou de responder a tudo: nem ping, nem portas. Não era código. Além disso, esse servidor não era da empresa, e quem o controlava teve acesso à sessão do WhatsApp e à chave de integração.

O que foi feito. O script `criar_evolution_ec2.sh` rodou no CloudShell e deixou pronto:

- servidor `emalog-evolution` na conta da empresa, sem chave SSH, protegido contra exclusão, com a porta da Evolution liberada só para o sistema;
- dados da Evolution num banco próprio dentro do mesmo PostgreSQL do RDS;
- senhas e chave guardadas criptografadas no Parameter Store da AWS;
- conta gratuita da Evolution Foundation ativada, com o painel aberto por alguns minutos apenas para o computador do responsável e fechado em seguida;
- instância de WhatsApp `emalog` criada e o sistema apontado para o servidor novo, com chave nova.

Confirmado. Depois de escanear o QR code, uma mensagem de teste enviada de um celular apareceu na fila da Central de Atendimento, com o histórico de atendimento registrado. O WhatsApp está funcionando ponta a ponta.

Duas observações do caminho, úteis se for preciso repetir: no painel da Evolution, a instância não deve ser criada à mão — o canal correto é o do QR code, não a API oficial da Meta, e o nome precisa ser exatamente o que o sistema espera. E o painel pede a chave global no login, que se lê do Parameter Store numa segunda aba do CloudShell, para não interromper o script.

4. Conversas ao lado do Kanban, e o Chatwoot desativado

A aba Conversas da Central de Contratação abria o Chatwoot em outra janela. Como o Chatwoot não será mais usado, essa aba passou a mostrar a própria Central de Atendimento: a mesma fila, o mesmo histórico e a mesma caixa de resposta, agora ao lado do Kanban e dos Cadastros EMA.

O painel virou um único trecho de código compartilhado, usado nos dois lugares: na página da Central de Atendimento e na aba da Contratação. Isso significa que toda correção futura vale nas duas telas ao mesmo tempo, sem risco de uma ficar diferente da outra. Os botões de som, avisos, atualizar e relatórios acompanharam o painel, para funcionarem nos dois lugares.

Do Chatwoot foram removidos: a rota que gerava o login dele, o código da tela antiga, a liberação que permitia exibir um site de terceiro dentro do sistema e as duas chaves de configuração. O sistema não lê mais nada do Chatwoot.

5. Testes

Nada foi entregue sem prova. Todos os testes rodam nos dois bancos, SQLite e PostgreSQL, como é a regra do projeto.

Teste	O que prova	Resultado
Regressão completa (25 linhas)	Fases 0 a 4 inteiras: fila, tempo real, bot, leitura, anexos, migrações, relatórios	Tudo OK
Trava contra SQLite	Recusa subir sem PostgreSQL na AWS, insiste antes de desistir, não cria banco vazio e não mostra senha no log	Tudo OK
Painel nas duas telas	33 verificações: painel presente uma vez em cada tela, Chatwoot fora, acesso por papel, Kanban intacto	Tudo OK
Conferência do pacote	54 verificações no zip de deploy, incluindo as mudanças de hoje	Tudo OK
Pacote em execução	O próprio zip subindo como na AWS, com PostgreSQL, e recusando subir sem banco	Tudo OK

Os dois testes novos também foram submetidos a sabotagem proposital do código que eles cobrem, para provar que reprovam quando deveriam: 10 e 6 verificações reprovadas, respectivamente.

6. O que falta fazer

Publicar o pacote. As mudanças dos itens 2 e 4 estão testadas, mas ainda não no ar. O pacote é `emalog-deploy-20260917-1617.zip`, em FONTE AWS RODANDO. Depois de publicar, conferir em `/conversas/api/status` que aparecem "dialetos": `"postgresql"` e `"trava_sqlite"`: `true`, e abrir a aba Conversas para atender uma conversa de teste.

Segurança e limpeza

Pendência	Por quê	Urgência
Trocar a senha do banco de produção	A linha completa de conexão apareceu num print enviado por conversa	Alta
Trocar a chave da Evolution	A chave nova também apareceu num print; ela dá controle total do WhatsApp	Alta

Pendência	Por quê	Urgência
Rotacionar os segredos que estão no histórico do git	Commits antigos levaram o arquivo de configuração com valores reais	Alta
Trocar o token da Twilio	Passou por conversa em sessão anterior	Média
Instalar HTTPS no ambiente	Hoje as conversas e os documentos dos motoristas trafegam em texto aberto	Média
Configurar o S3	Sem ele, fotos de CNH e CRLV recebidas não são guardadas	Média
Apagar as variáveis do Chatwoot do ambiente e desligar o servidor dele	O sistema já as ignora; o servidor é custo mensal sem uso	Baixa
Decidir o destino das rotas antigas de conversa da Contratação	Elas contornam a fila; nenhuma tela as usa hoje	Baixa

A lista completa, com detalhes técnicos, fica em `atendimento_conversas/PENDENCIAS.md`.

7. Uma recomendação

Duas credenciais se perderam hoje do mesmo jeito: em prints de tela enviados por conversa. Senha e chave nunca precisam sair da AWS — elas ficam guardadas no CloudShell e no Parameter Store, e o próprio sistema as lê de lá. Ao mandar print de um terminal, vale cortar o bloco final, que é onde essas linhas aparecem. Toda troca de credencial custa um reinício do sistema e uma nova rodada de conferência.